EgoBench: A Diagnostic Benchmark for Egocentric Annotation Pipelines

Ismael Duran

iaredur@stanford.edu

Abstract

Robots can pick up manipulation skills by watching first-person human video, the kind shot from a camera worn on the head or chest. Several recent systems already do this, but each one builds its own pipeline to turn raw video into training data, and each reports its own success numbers under its own conditions, so it is hard to say which part of the pipeline is actually doing the work. I read five of these systems closely and found they share the same four steps: tracking the hands, turning that motion into robot actions, writing language labels, and cutting the video into clips. EgoBench is my proposal for pulling those four steps apart and testing them one at a time. I hold the rest fixed, the source video, the base model, and the robot, and vary only the annotation step under test, then measure how the quality of each step carries through to how well the robot does the task. I pair five robot-side metrics with simpler proxy metrics for each step, lay out a protocol on the ALOHA 2 two-armed robot, and state the correlations I expect to see. This is a blueprint rather than a finished result. I have not run the experiments yet, so the numbers here are things I expect, not things I have measured.

1. Introduction

First-person video is video shot from a camera worn on a person, pointed out at their hands as they do things. There is a huge amount of it. Ego4D [1] alone holds thousands of hours of people doing everyday tasks across thousands of places. That got me interested in a simple question. If a robot could learn to handle objects just by watching humans in this kind of video, the usual bottleneck, collecting robot demonstrations one at a time by hand, would mostly go away. The catch is that the video is messy. Nobody labeled it, the camera shakes, and it is not obvious how to turn it into something a robot can train on.

So I started reading, and five recent systems convinced me this is doable. VITRA [2] builds a training set of a million clips automatically from raw human video. Masquerade [3] erases the human arm from each frame and paints a robot arm in its place, and reports a five to six times improvement over its baselines on two-handed kitchen tasks. CLAP [4] lines up a learned action code from human video with real robot motion. MAPLE [5] picks up dexterous hand skills by finding the moments where the hand touches an object. And a system from Physical Intelligence [6] shows that when the base model is already large and varied, the jump from human to robot can happen on its own, with almost no special labeling.

Reading these one after another, something nagged at me. Each paper builds its own pipeline to prepare the video, and each one reports plain task success rates measured under its own conditions, with different tasks, scenes, robots, and ways of scoring. So when one method beat another, I could not tell why. Was it how they tracked the hands, how they labeled the actions, the size of the dataset, or just a bigger model? EgoBench is my attempt to get at that. Rather than asking which whole system wins, it asks which single step of the pipeline actually matters for the robot, and how much.

2. Egocentric Annotation for Robot Learning

I review five recent methods and identify a shared pipeline structure (Figure 1): four annotation components that convert raw video into training data, and three training-stage components that determine how the model uses those annotations.

2.1. Reviewed Methods

CLAP [4] (Tsinghua/Astribot) collects demonstrations using Meta Quest VR hand tracking. The system learns a discrete latent action representation with a VD-VAE inverse dynamics model, aligned with robot proprioception through contrastive learning. High-level task language is generated by a VLM that decomposes tasks into sub-goals. The framework uses two VLA controllers: an autoregressive next-token prediction model and a Rectified Flow controller. Training combines several objectives, including Act-VAE, vector quantization (VQ), and SigLIP alignment, and draws on data from the AgiBot World, Astribot S1, and Ego4D datasets.

VITRA [2] (Tsinghua/Microsoft) automates the full pipeline. HaWoR extracts 6D wrist pose with MANO joint angles, a segmentation module produces activity boundaries, and GPT-4.1 auto-annotates language instructions. The architecture uses PaLiGemma2 with an action transformer and a diffusion action expert, trained on 1M episodes (26M frames) from Ego4D and EgoExo4D.

Masquerade [3] (Stanford) takes a video-editing approach. HaMeR extracts 21-keypoint hand skeletons, the human arms are inpainted out of the frame, and a rendered bimanual robot is overlaid to track the recovered end-effector trajectories within the original video. Language comes from Epic Kitchens labels with DistilBERT FiLM

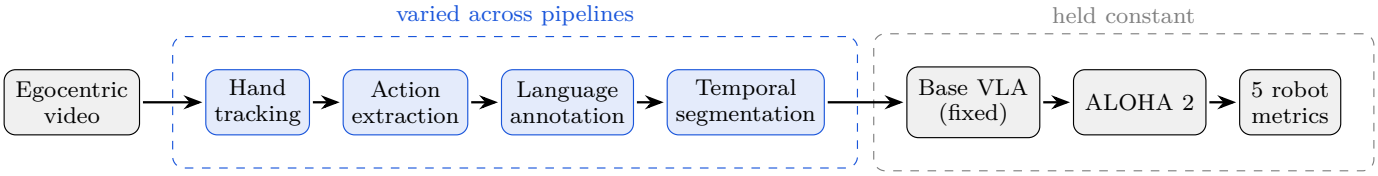

Fig. 1. EgoBench isolates the annotation pipeline. The source video, base model, and evaluation platform stay fixed, and only the four annotation components vary.

conditioning. A ViT encoder predicts future 2D keypoints, then co-trains with a diffusion policy on 50 real robot demonstrations per task.

MAPLE [5] (ETH Zürich) targets dexterous manipulation. HaMeR and TAPIR extract hand poses, and contact point backtracking identifies moments of hand-object interaction. The method is self-supervised with no language annotation, using contact cross-entropy, hand pose cross-entropy, and InfoNCE losses on Ego4D. Frozen encoder features feed a Transformer diffusion policy for a 17-DoF dexterous hand.

Emergence [6] (Physical Intelligence) shows that with a sufficiently diverse pretrained VLA ($\pi_{0.5}$), transfer emerges without explicit annotation. 3D hand positions from 3-view egocentric setups are treated as end-effector actions, and human data is co-finetuned with robot data in a 50-50 mix. I include this as a minimal-annotation baseline, to test whether structured annotation pipelines provide a measurable benefit over simple data mixing.

2.2. Annotation Components

I identify four annotation components shared across these pipelines.

Hand Tracking extracts 3D hand or end-effector pose from video frames. Methods range from VR teleoperation (CLAP) to learning-based regressors such as HaMeR (Masquerade, MAPLE) and HaWoR with MANO joints (VITRA), as well as point-tracking approaches like TAPIR (MAPLE).

Action Extraction derives robot-executable trajectories from tracked hand poses. Approaches include inverse dynamics latent encoding (CLAP), direct 3D trajectory regression (VITRA), contact point backtracking (MAPLE), and 3D end-effector projection with 2D waypoints (Masquerade).

Language Annotation generates task instructions and subtask labels. Strategies include VLM sub-goal decomposition (CLAP), GPT-4.1 visual prompting (VITRA), inherited metadata with learned conditioning (Masquerade), and no language annotation at all (MAPLE, Emergence).

Temporal Segmentation determines episode boundaries, that is, where one action ends and the next begins. VITRA segments atomic activities automatically. CLAP uses VLM sub-goal boundaries. Masquerade inherits boundaries from Epic Kitchens. MAPLE identifies contact frames and backprojects to find prediction frames. This component affects training signal granularity, since segmenting "pour water into cup" into three sub-actions produces a very different training distribution than treating it as one episode.

I distinguish these annotation components from training-stage variables, specifically the vision processing method, training objective, and co-training recipe, which EgoBench holds constant across all evaluated pipelines.

3. Gaps in Current Evaluation

Current evaluations of these methods share several limitations that EgoBench addresses.

First, inputs are not standardized. CLAP uses AgiBot and Ego4D. VITRA uses Ego4D and EgoExo4D. Masquerade uses Epic Kitchens. MAPLE uses Ego4D. Emergence uses proprietary internal data. Performance differences may reflect the data rather than the pipeline design.

Second, base models are not standardized. CLAP builds a custom architecture. VITRA finetunes PaLiGemma2. Emergence uses the proprietary $\pi_{0.5}$.

Third, evaluation metrics are simple. All five papers report binary task success, which hides why a method succeeds or fails.

Fourth, there are no component-level diagnostics. No existing work measures whether intrinsic hand tracking accuracy or language annotation quality predicts downstream robot performance.

Fifth, architecture sensitivity is untested. Some pipelines produce representations tied to specific model architectures. For example, CLAP targets autoregressive prediction, and feeding these representations to a diffusion-based policy may degrade performance. Masquerade modifies input data rather than action representations and would likely transfer more easily across architectures. This coupling has not been explored.

4. Proposed Benchmark: EgoBench

4.1. Hypotheses

H1: Intrinsic annotation quality, measured on established benchmarks, positively correlates with downstream robot performance.

H2: Pipelines with explicit language annotations yield higher subtask sequencing correctness than language-free approaches.

H3: Pipelines that modify input data (Masquerade) are more architecture-agnostic than those that modify action

representations (CLAP), as measured by performance variance across a base model.
H4: Component-level improvements, such as switching hand trackers, produce measurable gains in the corresponding metrics, allowing targeted pipeline optimization.

4.2. Standardization Protocol

I use Ego4D [1] as the egocentric source, since it is publicly available and already used by three of the five methods. Masquerade's pipeline, originally designed for Epic Kitchens, is adapted to Ego4D by replacing the inherited segment boundaries with VLM-based segmentation and regenerating the language labels.

The base VLA model is OpenVLA, an open-weight model not native to any of the reviewed papers. This removes any implicit bias toward a single pipeline. To test H3, I run a secondary evaluation with Octo, a diffusion-based model, to measure architecture sensitivity across models.

The training recipe is identical across all conditions, including learning rate, batch size, number of training steps, and a fixed human-to-robot data ratio of 70:30 for annotation pipelines and 50:50 for the Emergence baseline. The training objective is held constant using flow matching.

The evaluation platform is ALOHA 2, an open-source bimanual robot with a MuJoCo simulation model and RealSense D405 depth cameras. ALOHA 2 supports bimanual tasks, which Masquerade and CLAP require.

One limitation is that MAPLE targets 17-DoF dexterous hands, while ALOHA 2 uses parallel-jaw grippers.

4.3. Intrinsic Annotation Quality (Proxy Metrics)

Before running any robot experiments, I evaluate each pipeline's annotation components on established benchmarks, using held-out data that does not overlap with the robot evaluation tasks. This measures intrinsic quality independent of any downstream training, and it serves two purposes. First, the proxy scores act as diagnostic indicators: if a pipeline underperforms on robot tasks, they reveal which annotation stage is the bottleneck. Second, if strong correlations between proxy scores and downstream performance hold (H1), future researchers can evaluate new annotation methods using these metrics alone, without running a full robot experiment at every design iteration.

For hand tracking, I report Mean Per-Joint Position Error (MPJPE) on FreiHAND and InterHand2.6M, comparing HaMeR, HaWoR, TAPIR, and VR-based tracking. For action extraction, I measure trajectory smoothness (mean absolute jerk), physical plausibility (workspace constraint violation rate), and temporal consistency (frame-to-frame action variance on static scenes). For language annotation, I run a human evaluation on 200 held-out Ego4D clips, rating correctness, specificity, and temporal alignment on 1-to-5 scales. For temporal segmentation, I measure over- and under-segmentation rates against manually annotated ground truth on a 100-video subset, reported as F1 at 0.5-second tolerance.

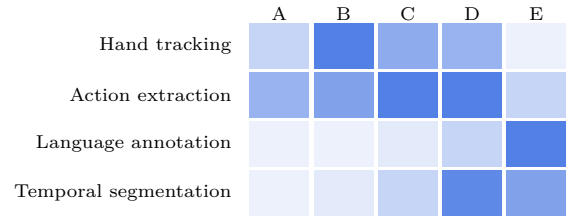

Fig. 2. Hypothesized component-to-outcome correlations (darker = stronger expected correlation). Columns: A visual robustness, B contact initiation, C grasp stability, D motion execution, E subtask sequencing.

4.4. Robot Metrics

I define five evaluation tasks on ALOHA 2, each designed to stress a specific manipulation capability. All evaluation scenes are out-of-distribution, with novel objects, backgrounds, and lighting conditions absent from the training data.

A. Visual Robustness evaluates four tabletop tasks (pick-and-place, pouring, sweeping, folding) under systematic perturbations including tablecloth color swaps, 30 percent lux changes, distractor objects, and 15-degree camera viewpoint shifts. The metric is the Area Under the Robustness Curve (AURC), capturing success rate degradation as perturbation intensity increases.

B. Contact Initiation Accuracy uses precision tasks such as grasping a mug by its handle or picking up a thin tool by the shaft. The 3D position of first gripper-object contact is recorded using depth cameras. The metric is mean Euclidean error in millimeters from the intended contact point.

C. Grasp Stability uses carry-and-place tasks such as transporting a plate or lifting and rotating a bottle. The metric combines slip rate (percentage of trials where the object is dropped) and normalized grip force variance estimated from servo motor readings.

D. Motion Execution Quality records full joint-space trajectories and computes a composite score from path length efficiency, mean jerk, and Dynamic Time Warping distance to expert teleoperation reference trajectories.

E. Subtask Sequencing Correctness uses multi-step tasks with ordering, such as setting a table where the plate must be placed first, then the fork on the left, then the knife on the right. The metric is the normalized Levenshtein edit distance between the executed subtask sequence and the ground-truth sequence, where 0 indicates perfect ordering and 1 indicates a completely incorrect sequence.

4.5. Correlation Analysis

The main analytical contribution of EgoBench is the component-to-outcome correlation. For each annotation

component c and each downstream metric m, I compute Spearman's rank correlation $\rho(c, m)$ across the four annotation pipelines. A strong correlation shows that improving a specific annotation component reliably improves a corresponding aspect of robot performance. A weak correlation would indicate that standard intrinsic benchmarks for that component do not predict transfer quality, which is itself a useful finding: it would suggest the field needs robotics-specific evaluation criteria. Figure 2 sketches the correlation structure EgoBench is designed to reveal.

5. Expected Results and Discussion

I expect a strong positive correlation between hand tracking quality and contact initiation accuracy. HaWoR (VITRA) provides 6D wrist pose with full MANO joint angles, which should yield finer end-effector targets than HaMeR's 21 keypoints. MAPLE's contact refinement should further improve precision, even though it uses HaMeR as its base tracker.

For language annotation and subtask sequencing, I expect pipelines with explicit language generation (VITRA, CLAP) to outperform language-free methods (MAPLE, Emergence) on sequencing correctness, supporting H2. Masquerade, which uses inherited dataset labels rather than generating its own, should fall between these two groups.

On architecture sensitivity, I expect CLAP's discrete codebook to show the largest performance drop when transferred from OpenVLA (autoregressive) to Octo (diffusion-based), supporting H3. Masquerade's data-editing approach changes the input distribution rather than the action representation and should transfer more easily across model families.

On the Emergence baseline, I expect the minimal-annotation approach to underperform the annotation-heavy pipelines when using the standardized OpenVLA, which is far smaller than the original $\pi_{0.5}$. This would quantify the value of structured annotation at smaller model scales and suggest that annotation effort complements model scale, mattering most when compute is limited.

For temporal segmentation, I expect that finer-grained segmentation (VITRA) will improve motion execution quality through more precise action boundaries. However, over-segmentation may hurt subtask sequencing by fragmenting coherent behaviors, a trade-off that EgoBench is designed to expose.

6. Future Directions

Natural extensions to EgoBench include additional egocentric datasets (EgoExo4D, Epic Kitchens), additional robot platforms (dexterous hands, mobile manipulators), and newer VLA architectures. The correlation framework extends beyond the five methods studied here: any future annotation pipeline can be evaluated within EgoBench by running the same proxy and downstream metrics.

References

[1] K. Grauman et al., "Ego4D: Around the world in 3,000 hours of egocentric video," in Proc. IEEE/CVF CVPR, 2022, pp. 18995–19012.

[2] Q. Li et al., "Scalable VLA model pretraining for robotic manipulation with real-life human activity videos," arXiv:2510.21571, 2025.

[3] M. Lepert, J. Fang, and J. Bohg, "Masquerade: Learning from in-the-wild human videos using data-editing," arXiv:2508.09976, 2025.

[4] C. Zhang et al., "CLAP: Contrastive latent action pretraining for learning VLA models from human videos," arXiv:2601.04061, 2026.

[5] A. Gavryushin et al., "MAPLE: Encoding dexterous robotic manipulation priors learned from egocentric videos," arXiv:2504.06084, 2025.

[6] S. Kareer et al., "Emergence of human to robot transfer in vision-language-action models," arXiv:2512.22414, 2025.